

```
In [ ]: !pip install fastapi uvicorn sqlalchemy passlib[bcrypt] python-jose python-multipart
```

```
In [ ]: import os

os.makedirs("movie_api", exist_ok=True)

files = [
    "movie_api/database.py",
    "movie_api/models.py",
    "movie_api/schemas.py",
    "movie_api/auth.py",
    "movie_api/main.py"
]

for file in files:
    open(file, "w").close()

print("Project Created")
```

```
In [ ]: from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///./movies.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()
```

```
In [ ]: from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base

DATABASE_URL = "sqlite:///./movies.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

Base = declarative_base()

print("Database Connected Successfully")
```

```
In [79]: from sqlalchemy import Column, Integer, String, Boolean, ForeignKey, DateTime
from sqlalchemy.orm import declarative_base, relationship
from datetime import datetime

Base = declarative_base()

class UserDB(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True)
    password = Column(String)
    role = Column(String, default="consumer")

    rentals = relationship("Rental", back_populates="user")

class Movie(Base):
    __tablename__ = "movies"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    genre = Column(String)
    available = Column(Boolean, default=True)

    rentals = relationship("Rental", back_populates="movie")

class Rental(Base):
    __tablename__ = "rentals"

    id = Column(Integer, primary_key=True, index=True)

    user_id = Column(Integer, ForeignKey("users.id"))
    movie_id = Column(Integer, ForeignKey("movies.id"))

    rent_date = Column(DateTime, default=datetime.utcnow)
    return_date = Column(DateTime, nullable=True)

    user = relationship("User", back_populates="rentals")
    movie = relationship("Movie", back_populates="rentals")

print("Models Created Successfully")
```

Models Created Successfully

```
In [ ]: from pydantic import BaseModel
class Usercreate(BaseModel):
    username : str
    passwors : str
class Moviecreate(BaseModel):
    title:str
    genre:str
```

```
In [ ]: from passlib.context import CryptContext
pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)
def hash_password(password):
    return pwd_context.verify(plain,hashed)
def verify_password(plain, hashed):
    return pwd_context.verify(plain,hashed)
```

```
In [ ]: from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    username: str
    password: str

@app.get("/")
def home():
    return {"message": "Movie API Running"}

@app.get("/test")
def test():
    return {"status": "ok"}

@app.post("/register")
def register(user: UserCreate):
    return {
        "message": "User registered successfully",
        "username": user.username
    }
```

```
In [ ]: with open("movie_api/main.py", "r") as f:
    print(f.read())
```

```
In [ ]: %%writefile movie_api/main.py

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    username: str
    password: str

@app.get("/")
def home():
    return {"message": "Movie API Running"}

@app.get("/test")
def test():
```

```

        return {"status": "ok"}

@app.post("/register")
def register(user: UserCreate):
    return {
        "message": "User registered successfully",
        "username": user.username
    }

```

```
In [ ]: !pip install bcrypt
```

```
In [ ]: from pydantic import BaseModel
class UserCreate (BaseModel):
    username: str
    password: str
```

```
In [ ]: from passlib.context import CryptContext
pwd_context = CryptContext
schemes=["bcrypt"],
deprecated="auto"

def hash_password(password):
    return pwd_context.hash(password)
```

```
In [ ]: users = []
```

```
In [ ]: import os
print(os.getcwd())
```

```
In [ ]: with open("movie_api/main.py", "r") as f:
    print(f.read())
```

```
In [ ]: import os
print(os.getcwd())
```

```
In [ ]: print("Hello")
```

```
In [ ]: import os
print(os.getcwd())
```

```
In [ ]: with open("movie_api/main.py", "r") as f:
    print(f.read())
```

```
In [73]: %%writefile movie_api/main.py

from fastapi import FastAPI, Form, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel
from jose import jwt, JWTError

from movie_api.auth import create_access_token
from movie_api.database import SessionLocal
from movie_api.models import MovieDB, RentalDB, UserDB
```

```
import hashlib

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

SECRET_KEY = "mysecretkey"
ALGORITHM = "HS256"

class UserCreate(BaseModel):
    username: str
    password: str

class Movie(BaseModel):
    id: int
    title: str
    genre: str
    year: int

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )

        username = payload.get("sub")

        if username is None:
            raise HTTPException(
                status_code=401,
                detail="Invalid token"
            )

        return username

    except JWTError:
        raise HTTPException(
            status_code=401,
            detail="Invalid token"
```

```
)

@app.get("/")
def home():
    return {"message": "Movie API Running"}

@app.get("/test")
def test():
    return {"status": "ok"}

@app.post("/register")
def register(user: UserCreate):

    db = SessionLocal()

    existing_user = db.query(UserDB).filter(
        UserDB.username == user.username
    ).first()

    if existing_user:
        return {"message": "Username already exists"}

    hashed_password = hash_password(user.password)

    db_user = UserDB(
        username=user.username,
        password=hashed_password
    )

    db.add(db_user)
    db.commit()

    return {
        "message": "User registered successfully"
    }

@app.post("/token")
def login(
    username: str = Form(...),
    password: str = Form(...)
):

    db = SessionLocal()

    user = db.query(UserDB).filter(
        UserDB.username == username
    ).first()

    if not user:
        raise HTTPException(
            status_code=401,
```

```

        detail="Invalid username or password"
    )

    hashed_password = hash_password(password)

    if user.password != hashed_password:
        raise HTTPException(
            status_code=401,
            detail="Invalid username or password"
        )

    access_token = create_access_token(
        data={"sub": username}
    )

    return {
        "access_token": access_token,
        "token_type": "bearer"
    }

@app.post("/dbmovies")
def add_db_movie(movie: Movie):

    db = SessionLocal()

    db_movie = MovieDB(
        title=movie.title,
        genre=movie.genre,
        year=movie.year
    )

    db.add(db_movie)
    db.commit()
    db.refresh(db_movie)

    return {
        "message": "Movie saved in database",
        "id": db_movie.id
    }

@app.get("/dbmovies")
def get_db_movies():

    db = SessionLocal()

    movies = db.query(MovieDB).all()

    return movies

@app.put("/dbmovies/{movie_id}")

```

```
def update_db_movie(movie_id: int, movie: Movie):

    db = SessionLocal()

    db_movie = db.query(MovieDB).filter(
        MovieDB.id == movie_id
    ).first()

    if not db_movie:
        return {"message": "Movie not found"}

    db_movie.title = movie.title
    db_movie.genre = movie.genre
    db_movie.year = movie.year

    db.commit()

    return {
        "message": "Movie updated successfully"
    }

@app.delete("/dbmovies/{movie_id}")
def delete_db_movie(movie_id: int):

    db = SessionLocal()

    db_movie = db.query(MovieDB).filter(
        MovieDB.id == movie_id
    ).first()

    if not db_movie:
        return {"message": "Movie not found"}

    db.delete(db_movie)
    db.commit()

    return{
        "message": "Movie Deleted successfully"
    }

@app.post("/dbmovies/{movie_id}/rent")
def rent_movie(
    movie_id: int,
    current_user: str = Depends(get_current_user)
):

    db = SessionLocal()

    movie = db.query(MovieDB).filter(
        MovieDB.id == movie_id
    ).first()

    if not movie:
        return {"message": "Movie not found"}
```



```

    if not movie.available:
        return {"message": "Movie already rented"}

    movie.available = False

    rental = RentalDB(
        username=current_user,
        movie_id=movie_id,
        status="rented"
    )

    db.add(rental)
    db.commit()

    return {
        "message": "Movie rented successfully"
    }

@app.get("/profile")
def profile(
    current_user: str = Depends(get_current_user)
):
    return {
        "message": f"Welcome {current_user}"
    }
@app.get("/users/me/rentals")
def my_rentals(
    current_user: str = Depends(get_current_user)
):

    db = SessionLocal()

    rentals = db.query(RentalDB).filter(
        RentalDB.username == current_user,
        RentalDB.status == "rented"
    ).all()

    return rentals

```

Overwriting movie_api/main.py

```
In [ ]: with open("movie_api/main.py", "r") as f:
        print(f.read())
```

```
In [ ]: !pip install passlib[bcrypt] bcrypt
```

```
In [ ]: print("LOADED CORRECT MAIN.PY")
```

```
In [ ]: !python -m uvicorn movie_api.main:app --reload
```

```
In [ ]: %%writefile movie_api/auth.py

from passlib.context import CryptContext

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)

def hash_password(password: str):
    return pwd_context.hash(password)
```

```
In [ ]: from passlib.context import CryptContext

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str):
    return pwd_context.verify(plain_password, hashed_password)
```

```
In [ ]: hash_password("123456")
```

```
In [ ]: hash_password("123456")
```

```
In [ ]: from movie_api.auth import hash_password

print(hash_password("123456"))
```

```
In [ ]: !pip uninstall -y bcrypt
!pip install bcrypt==4.0.1
```

```
In [ ]: from movie_api.auth import hash_password

print(hash_password("123456"))
```

```
In [ ]: print("Hello")
```

```
In [ ]: !python -m uvicorn movie_api.main:app --reload
```

```
In [ ]: import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

print(hash_password("123456"))
```

```
In [ ]: import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

print(hash_password("123456"))
```

```
In [ ]: import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

print(hash_password("123456"))
```

```
In [ ]: !pip install bcrypt
```

```
In [ ]: import bcrypt

def hash_password(password):
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password.encode(), salt).decode()

print(hash_password("123456"))
```

```
In [ ]: pip install python-jose[cryptography] python-multipart
```

```
In [ ]: %%writefile movie_api/auth.py

from jose import jwt
from datetime import datetime, timedelta

SECRET_KEY = "mysecretkey"
ALGORITHM = "HS256"

def create_access_token(data: dict):
    to_encode = data.copy()

    expire = datetime.utcnow() + timedelta(minutes=30)
    to_encode.update({"exp": expire})

    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

```
In [66]: from movie_api.main import app

for route in app.routes:
    print(route.path)
```

```

/openapi.json
/docs
/docs/oauth2-redirect
/redoc
/
/test
/register
/token
/dbmovies
/dbmovies
/dbmovies/{movie_id}
/dbmovies/{movie_id}
/dbmovies/{movie_id}/rent
/profile

```

In [5]: `from movie_api.main import app`

```

for route in app.routes:
    print(route.path)

```

```

/openapi.json
/docs
/docs/oauth2-redirect
/redoc
/
/test
/register
/token
/movies
/movies
/movie/{movie_id}

```

In [2]: `import importlib`
`import movie_api.main`
`importlib.reload(movie_api.main)`
`for route in movie_api.main.app.routes:`
 `print(route.path)`

```

/openapi.json
/docs
/docs/oauth2-redirect
/redoc
/
/test
/register
/token
/dbmovies
/dbmovies
/dbmovies/{movie_id}
/dbmovies/{movie_id}
/profile

```

In [16]: `for route in movie_api.main.app.routes:`
 `print(route.path, route.methods)`

```

/openapi.json {'GET', 'HEAD'}
/docs {'GET', 'HEAD'}
/docs/oauth2-redirect {'GET', 'HEAD'}
/redoc {'GET', 'HEAD'}
/ {'GET'}
/test {'GET'}
/register {'POST'}
/token {'POST'}
/movies {'POST'}
/movies {'GET'}
/movies/{movie_id} {'PUT'}
/movies/{movie_id} {'DELETE'}

```

In [17]: `!pip install sqlalchemy`

```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: sqlalchemy in c:\programdata\anaconda3\lib\site-packa
ges (2.0.43)
Requirement already satisfied: greenlet>=1 in c:\programdata\anaconda3\lib\site-pack
ages (from sqlalchemy) (3.2.4)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\programdata\anaconda3
\lib\site-packages (from sqlalchemy) (4.15.0)

```

In [53]: `%%writefile movie_api/database.py`

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from movie_api.models import Base

DATABASE_URL = "sqlite:///./movies.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base.metadata.create_all(bind=engine)

```

Overwriting movie_api/database.py

In [2]: `from movie_api.database import engine`
`print("Database Connected Successfully")`

Database Connected Successfully

In [47]: `%%writefile movie_api/models.py`

```

from sqlalchemy import Column, Integer, String, Boolean
from sqlalchemy.orm import declarative_base

Base = declarative_base()

```

```

class MovieDB(Base):
    __tablename__ = "movies"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    genre = Column(String)
    year = Column(Integer)

    available = Column(Boolean, default=True)

class RentalDB(Base):
    __tablename__ = "rentals"

    id = Column(Integer, primary_key=True, index=True)

    username = Column(String)
    movie_id = Column(Integer)

    status = Column(String, default="rented")

```

Overwriting movie_api/models.py

```

In [1]: from movie_api.database import engine
        from movie_api.models import Base
        Base.metadata.create_all(bind=engine)
        print("Movie table created successfully")

```

Movie table created successfully

```

In [13]: from movie_api.models import MovieDB
         print("Model Loaded Successfully")

```

Model Loaded Successfully

```

In [23]: %%writefile movie_api/schemas.py

        from pydantic import BaseModel

        class MovieCreate(BaseModel):
            title: str
            genre: str
            year: int

        class MovieResponse(MovieCreate):
            id: int

        class Config:
            from_attributes = True

```

Overwriting movie_api/schemas.py

```

In [24]: from movie_api.schemas import MovieCreate
         print("Schemas Loaded Successfully")

```

Schemas Loaded Successfully

```
In [26]: from movie_api.database import SessionLocal

db= SessionLocal()

print("Database Session Created")
```

Database Session Created

```
In [27]: movie= []
```

```
In [28]: from movie_api.database import SessionLocal
from movie_api.models import MovieDB

db = SessionLocal()

movie = MovieDB(
    title="Animal",
    genre="Action",
    year=2023
)

db.add(movie)
db.commit()

print("Movie Saved Successfully")
```

Movie Saved Successfully

```
In [30]: from movie_api.database import SessionLocal
from movie_api.models import MovieDB

db = SessionLocal()

movies = db.query(MovieDB).all()

for movie in movies:
    print(movie.id, movie.title, movie.genre, movie.year)
```

1 Animal Action 2023

```
In [38]: import importlib
import movie_api.main

importlib.reload(movie_api.main)

print("main.py reloaded")
```

main.py reloaded

```
In [39]: from jose import jwt

SECRET_KEY = "mysecretkey"
ALGORITHM = "HS256"

def create_access_token(data: dict):
    return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)
```

```
In [41]: from movie_api.models import MovieDB

print("Model Updated Successfully")
```

Model Updated Successfully

```
In [52]: from movie_api.models import MovieDB, RentalDB

print("MovieDB Loaded")
print("RentalDB Loaded")
```

MovieDB Loaded
RentalDB Loaded

```
In [54]: from movie_api.database import SessionLocal

db = SessionLocal()

print("Database Ready")
```

Database Ready

```
In [2]: import os

if os.path.exists("movies.db"):
    os.remove("movies.db")

print("Old database deleted")
```

Old database deleted

```
In [4]: import fastapi
print(fastapi.__version__)
```

0.137.1

```
In [1]: exit()
```

```
In [16]: import os
print(os.path.abspath("movie_api/main.py"))
```

C:\Users\DELL\movie_api\main.py

```
In [54]: with open("movie_api/main.py", "r") as f:
        content = f.read()

content = content.replace(
    """class UserCreate(BaseModel):
    username: str
    password: str""",
    """class UserCreate(BaseModel):
        username: str
        password: str"""
)

content = content.replace(
    """class Movie(BaseModel):
    id: int
```



```

title: str
genre: str
year: int""",
"""class Movie(BaseModel):
    id: int
    title: str
    genre: str
    year: int"""
)

with open("movie_api/main.py", "w") as f:
    f.write(content)

print("File updated")

```

File updated

```

In [76]: from movie_api.main import app
        print("Main Loaded Successfully")

```

Main Loaded Successfully

```

In [75]: from movie_api.main import app
        print("Main Loaded")

```

Main Loaded

```

In [81]: %%writefile movie_api/models.py

        from sqlalchemy import Column, Integer, String, Boolean
        from sqlalchemy.orm import declarative_base

        Base = declarative_base()

        class UserDB(Base):
            __tablename__ = "users"

            id = Column(Integer, primary_key=True, index=True)
            username = Column(String, unique=True, index=True)
            password = Column(String)

        class MovieDB(Base):
            __tablename__ = "movies"

            id = Column(Integer, primary_key=True, index=True)
            title = Column(String, index=True)
            genre = Column(String)
            year = Column(Integer)

            available = Column(Boolean, default=True)

        class RentalDB(Base):
            __tablename__ = "rentals"

            id = Column(Integer, primary_key=True, index=True)

            username = Column(String)

```

```

movie_id = Column(Integer)

status = Column(String, default="rented")

```

Overwriting movie_api/models.py

```

In [88]: from movie_api.models import UserDB
print("UserDB Loaded")

```

UserDB Loaded

```

In [85]: import importlib
import movie_api.models

importlib.reload(movie_api.models)

from movie_api.models import UserDB

print("UserDB Loaded")

```

UserDB Loaded

```

In [86]: with open(movie_api.models.__file__, "r") as f:
print(f.read())

```

```

from sqlalchemy import Column, Integer, String, Boolean
from sqlalchemy.orm import declarative_base

```

```
Base = declarative_base()
```

```

class UserDB(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True)
    password = Column(String)

```

```

class MovieDB(Base):
    __tablename__ = "movies"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    genre = Column(String)
    year = Column(Integer)

    available = Column(Boolean, default=True)

```

```

class RentalDB(Base):
    __tablename__ = "rentals"

    id = Column(Integer, primary_key=True, index=True)

    username = Column(String)
    movie_id = Column(Integer)

    status = Column(String, default="rented")

```

```
In [87]: import movie_api.models
import importlib

print("Loaded from:", movie_api.models.__file__)

importlib.reload(movie_api.models)

print("Has UserDB:", hasattr(movie_api.models, "UserDB"))

from movie_api.models import UserDB

print("UserDB Loaded Successfully")
```

Loaded from: C:\Users\DELL\movie_api\models.py

Has UserDB: True

UserDB Loaded Successfully

In []: